PAK-AUSTRIA FACHHOCHSCHULE:
INSTITUTE OF APPLIED SCIENCES AND TECHNOLOGY

AF-IAST
Skilling Pakistan

TrackIT

Bug Tracking & Issue Management System

Software Construction & Development

Team members & roles:

Members	*Roles*
Shayan Ullah B23F0001SE03	Project Manager & Back-end Developer
Syeda Sabeeka Shah B23F0001SE001	Front-end Developer
Khadija Khalid B23F0013SE045	Database Designer
Abdul Aziz B23F0001SE038	QA & Testing Engineer
Muhammad Rehan B23F0001SE011	DevOps & Documentation Lead.

1. Introduction

1.1 Project Overview

TrackIT was developed as part of the COMP-370 Software Construction and Development (SCD) Engineering Senior Project (ESP). The system replaces disjointed tracking methods — such as spreadsheets or email chains — with a unified, role-based platform. Built on a robust client-server architecture, TrackIT features a dynamic React.js frontend and a secure Spring Boot backend, enabling teams to seamlessly log bugs, attach visual evidence, assign developers, and monitor issue resolution in real time through an interactive dashboard.

1.2 Purpose & Objectives

The primary purpose of TrackIT is to provide a structured, transparent, and efficient workflow for handling software anomalies. The project is guided by the following core objectives:

- **Centralize Issue Management:** Establish a single source of truth where all bugs, feature requests, and system anomalies are logged, ensuring no issue is overlooked.
- **Enforce Role-Based Accountability:** Implement strict Role-Based Access Control (RBAC) separating users into Admins, Developers, and Reporters to ensure proper task routing and access restriction.
- **Enhance Team Communication:** Provide integrated commenting threads and a real-time notification system to facilitate clear communication between reporters and developers.
- **Provide Real-Time Analytics:** Equip administrators with an overarching health dashboard that visualizes project progress, resolution times, and the current distribution of open vs. closed issues.
- **Ensure Robust Security:** Secure all data transactions using stateless JSON Web Tokens (JWT) and enforce rigorous data validation to prevent common web vulnerabilities.

1.3 Scope of the Project

The scope of TrackIT encompasses the complete lifecycle of a software bug, from initial discovery to final resolution. The system is designed to support small to medium-sized development teams.

In-Scope Features

- **Authentication & Authorization:** Secure user registration, login, and JWT-based session management across three distinct roles.
- **Issue Lifecycle Management:** Full CRUD operations on issues, including metadata such as priority (Low, Medium, High, Critical), status (Open, In Progress, Resolved, Closed), and rich-text descriptions.
- **File Attachments:** A secure filesystem utility allowing users to upload and download screenshots or error logs directly attached to specific tickets.

- **Collaboration Tools:** An interactive commenting system tied to individual issues, alongside an automated bell-based notification system that alerts users to updates.
- **Role-Specific Dashboards:** Custom user interfaces that dynamically adapt based on the user's role (e.g., Developers see "My Assigned Tasks"; Admins see "Global Project Analytics").

Out-of-Scope Features

The following features were excluded to maintain focus on core assignment requirements but remain candidates for future iterations:

- Third-party integrations (e.g., GitHub commit tracking, Slack integration)
- Email or SMS push notifications
- Billing, subscription management, or multi-tenant SaaS structures
- Admin-approved new user registrations

3. How TrackIT Works

TrackIT operates on a classic Client-Server request-response cycle. The core philosophy of the system is state-driven issue management combined with role-based access control. When a user interacts with the system, the React frontend translates their actions into JSON payloads, which are transmitted via HTTP requests to the Spring Boot backend. The backend acts as an authoritative gatekeeper: it first verifies the user's identity via a JWT token, confirms the correct role for the requested action, and then executes the necessary business logic. Data is persisted in a MySQL database, while media files (such as error screenshots) are stored in the server's local filesystem. Whenever data changes — such as an issue being updated — the system automatically triggers a notification observer to alert relevant parties.

3.1 System Architecture

1. Frontend Layer (React SPA)

The React Single Page Application (SPA) serves as the user-facing layer. Users — Admins, Developers, and Reporters — interact with the system through UI components including dashboards, forms, and pages. The frontend leverages the React Context API for global state management and the Axios HTTP Client to send REST API requests with JWT authentication headers.

2. Security Layer

Every incoming request passes through the Spring Security & JWT Filter before reaching any business logic. This layer authenticates users, validates JWT tokens, and enforces Role-Based Access Control (RBAC). Only authorized users can access protected resources and execute specific operations.

3. Backend Layer (Spring Boot Monolith)

The Spring Boot backend is divided into multiple layers:

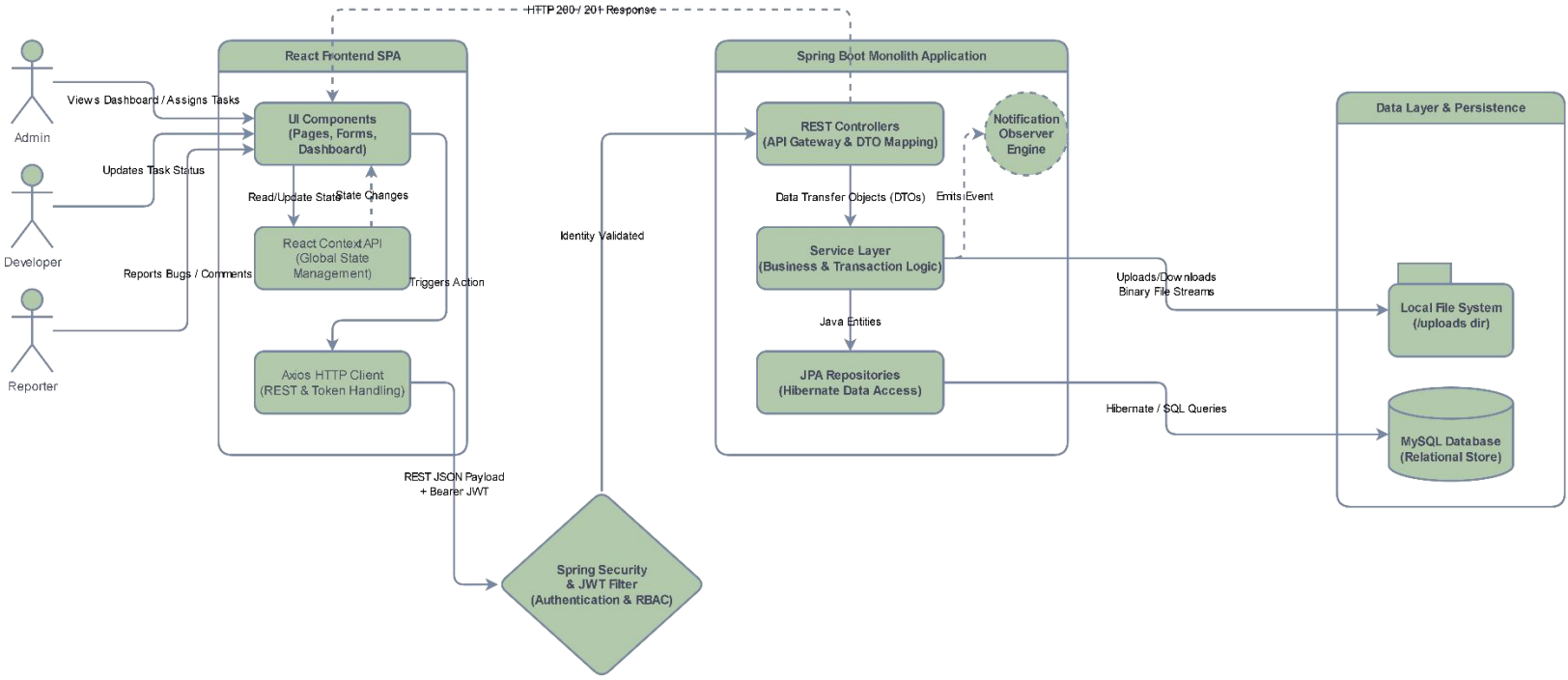
- REST Controllers — handle incoming API requests and return responses
- Service Layer — contains business logic and transaction handling
- JPA Repositories — manage database operations using Hibernate

The backend also includes a Notification Observer Engine, which automatically generates notifications when important events occur, such as task assignments or status updates.

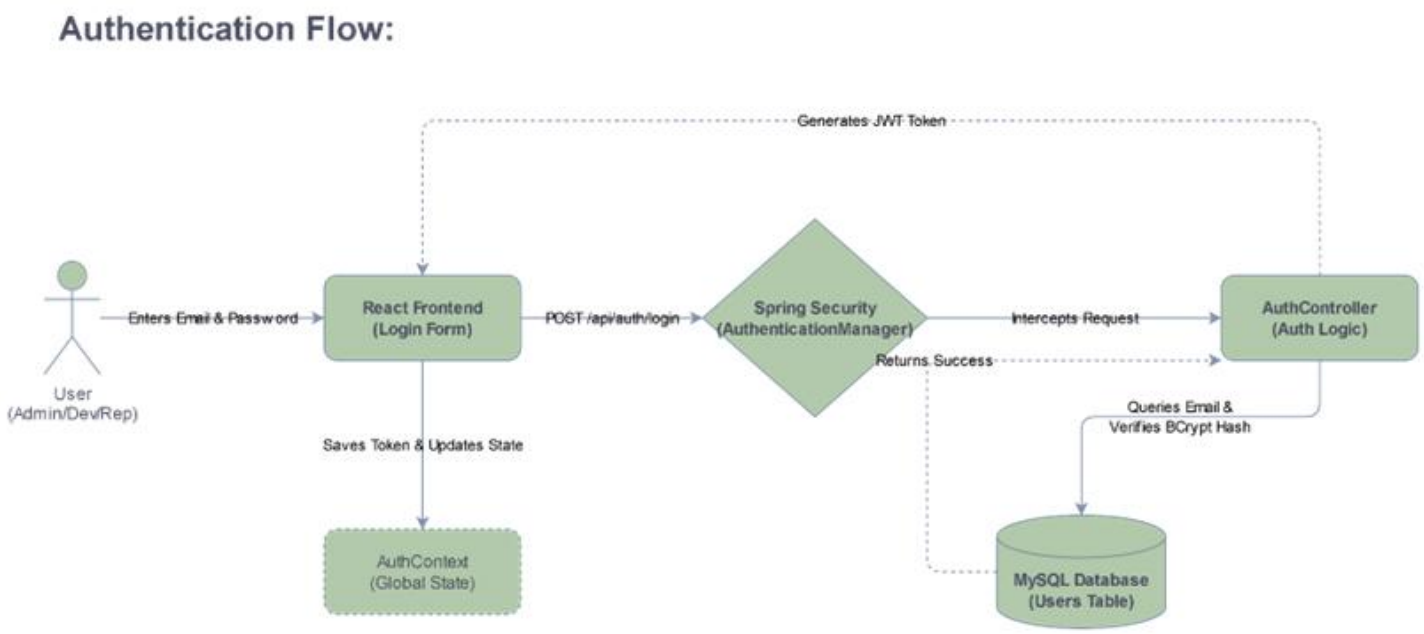
4. Data & Persistence Layer

Structured application data is stored in a MySQL relational database, while uploaded files (screenshots and PDFs) are saved to the server's local filesystem (/uploads directory).

3.1.1 System Architecture Diagram:



3.1.2 Flow diagrams:



3.2 Authentication Flow

The authentication process proceeds through the following steps:

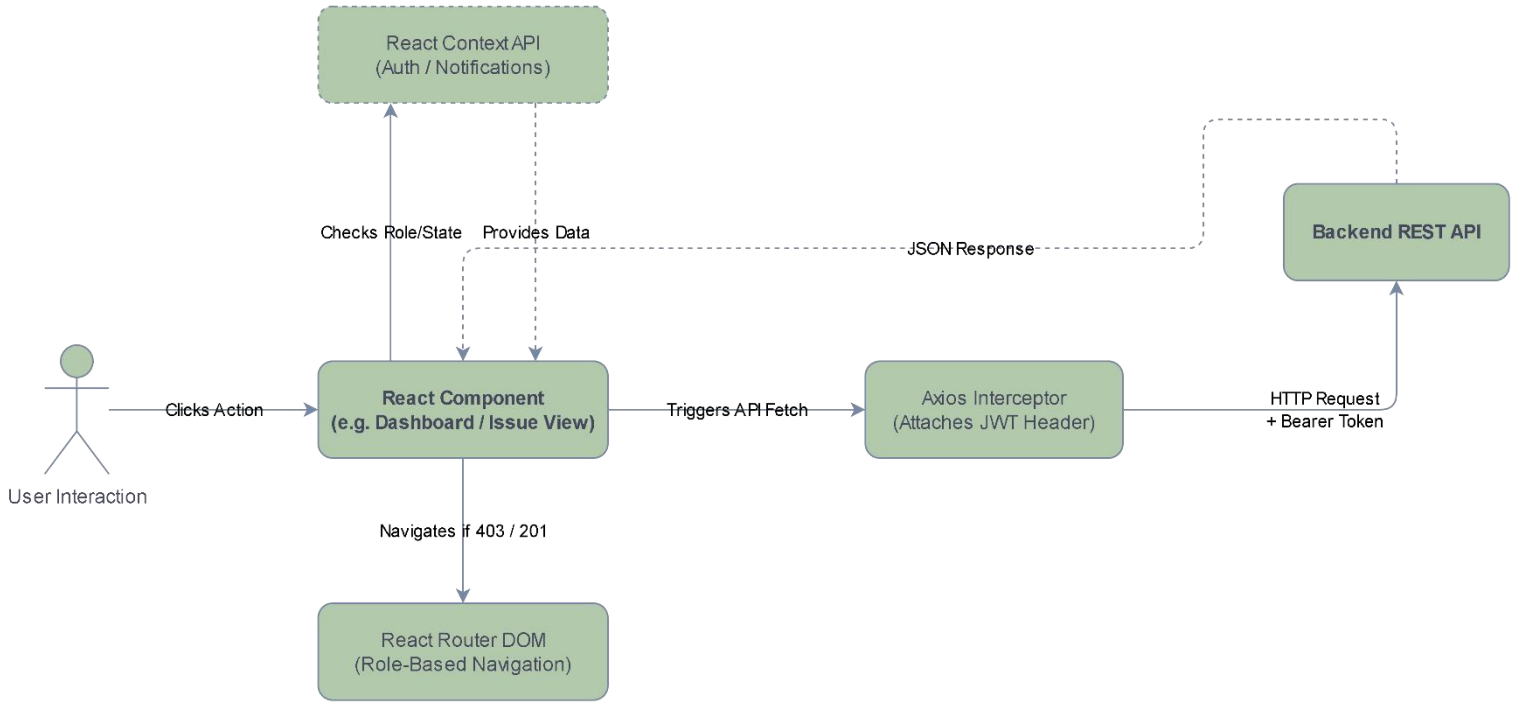
- **Initiation:** The user inputs credentials into the React frontend.
- **Transmission:** The frontend sends a POST request containing a JSON payload to the backend API endpoint POST /api/auth/login.
- **Verification:** The Spring Security AuthenticationManager intercepts the request, queries the database, and compares the provided password against the stored BCrypt hash.
- **Token Generation:** If valid, the backend generates a cryptographically signed JWT and returns it to the client.
- **State Update:** React stores the token in localStorage, updates the AuthContext, marks the user as authenticated, and unlocks role-specific routes.

3.3 Frontend-to-Backend Data Pipeline

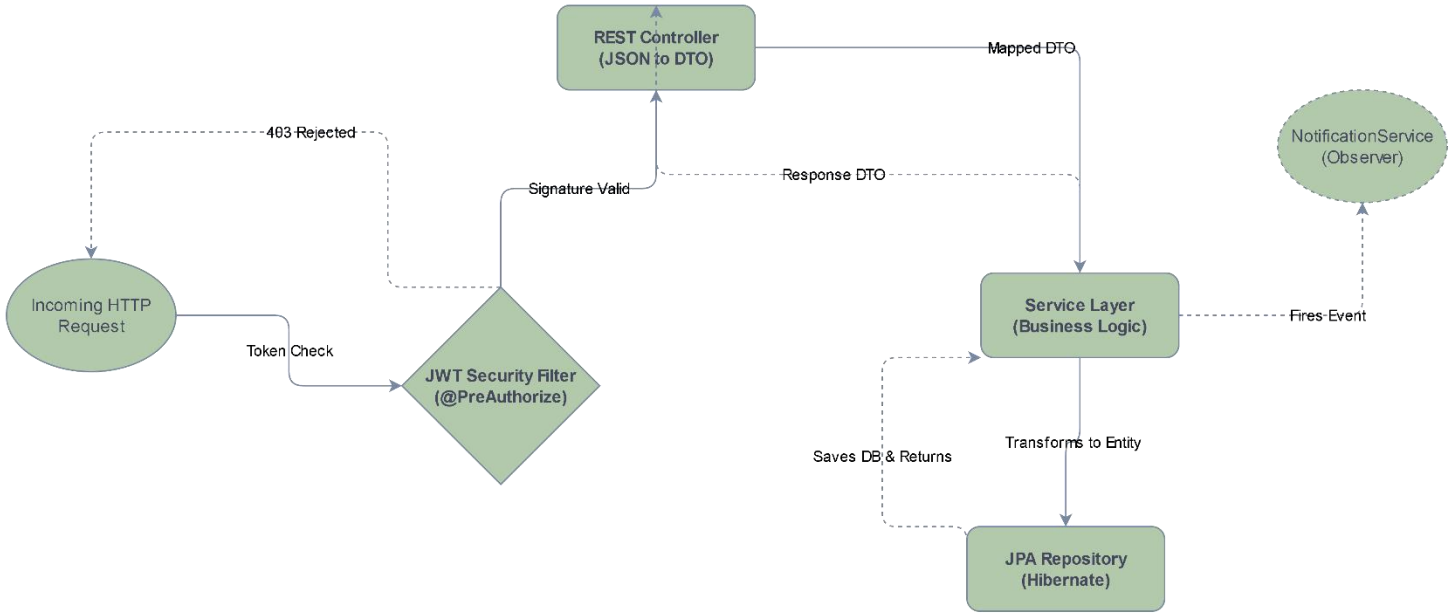
- **Frontend Trigger:** A user triggers an action (e.g., clicks "Save Issue"). The React component calls an API function.
- **Axios Interception:** Before the request leaves the browser, an Axios Interceptor automatically attaches the JWT to the Authorization header.
- **Security Gateway:** Spring Boot's @PreAuthorize filter intercepts the request and verifies the user's role, rejecting unauthorized access (e.g., a REPORTER attempting to reach an ADMIN endpoint).

- **Business Logic & Persistence:** The Controller maps the JSON payload to a DTO; the Service applies business rules; the JPA Repository executes the Hibernate SQL query to persist the data.
- **UI Re-render:** The backend returns HTTP 200 OK and React updates UI state without a full page refresh.

Frontend-Data Flow:



Backend-Data Flow:

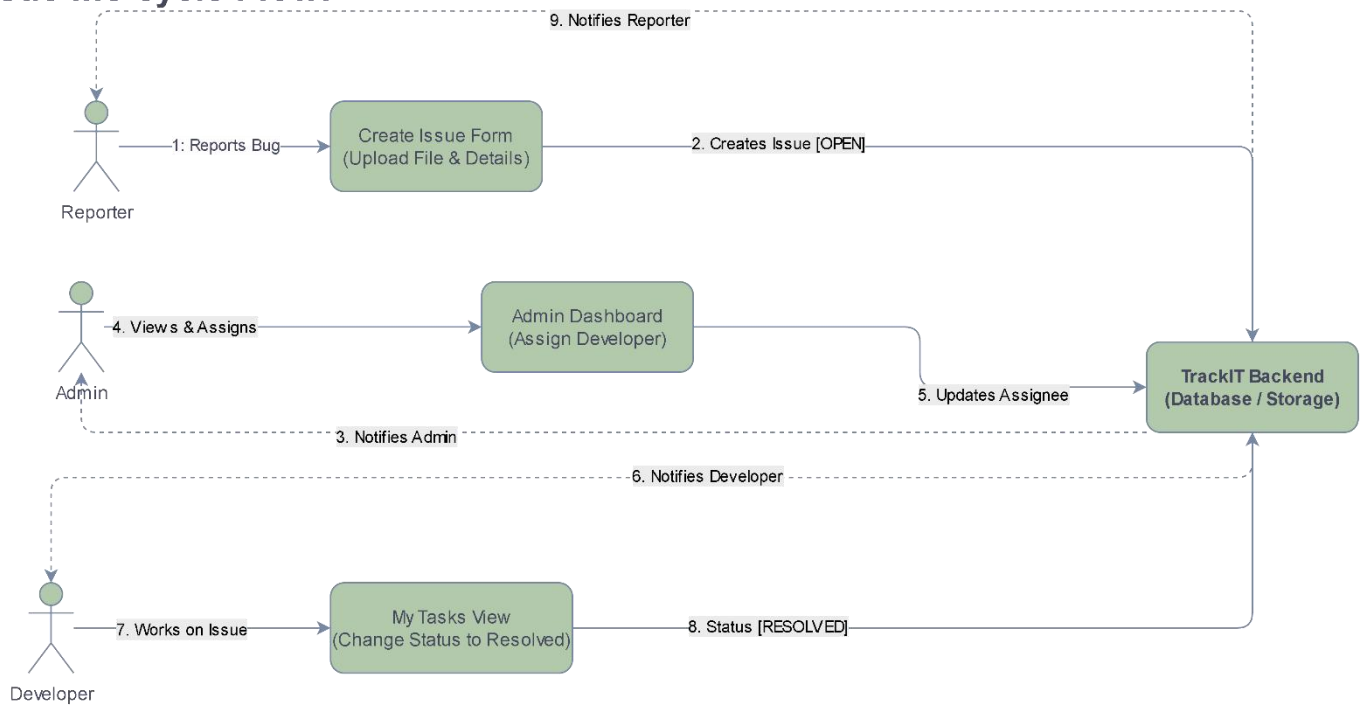


3.4 End-to-End User Journey

The following lifecycle demonstrates how TrackIT functions as a complete ecosystem — from bug discovery to final resolution:

- **Login & Routing:** A Reporter logs in. The frontend reads their JWT role and automatically routes them to the "My Reports" dashboard.
- **Issue Creation:** The Reporter encounters a system crash, clicks "Report Issue", fills out the form, attaches a screenshot, and submits. The backend saves the issue and triggers the NotificationService.
- **Triage & Assignment:** An Admin logs in, views the Global Dashboard, finds the new issue in their notification feed, and assigns it to a specific Developer.
- **Resolution:** The Developer receives a notification, navigates to their "My Tasks" view, fixes the bug, updates the issue status to RESOLVED, and leaves a comment explaining the fix.
- **Closure:** The system instantly notifies the Reporter that their bug is resolved. The Reporter verifies the fix, leaves a closing comment, and the lifecycle is complete.

Issue life-cycle Flow:



4. System Operation & API Data Flow

TrackIT operates on a RESTful, stateless Client-Server model. Every user action on the React frontend triggers an Axios HTTP request. Because the system is stateless, the backend does not maintain session state; instead, the frontend proves user identity on every request by attaching a JSON Web Token.

4.1 Authentication Flow (Security Gateway)

- **Frontend Call:** The user enters credentials. React packages them and fires POST /api/auth/login.
- **Backend Resolution:** The AuthenticationManager queries the users table by email, compares the password against the BCrypt hash, and — if valid — generates a signed JWT containing the user's ID and role (ADMIN, DEVELOPER, or REPORTER).
- **Response:** HTTP 200 OK is returned with the JWT and user details. React saves the token in localStorage and updates the global AuthContext.
- **Subsequent Calls:** React's Axios Interceptor automatically injects Authorization: Bearer <token> into all future HTTP headers.

4.2 Issue Management Flow

A. Creating a New Bug Report

- **Frontend Call:** A Reporter fills out a bug report and attaches a screenshot. Because a file is included, React converts the payload to multipart/form-data and fires POST /api/issues.
- **Backend Resolution:** The Security Filter Chain validates the JWT and confirms the REPORTER or ADMIN role. IssueController passes the file to FileStorageUtil, which streams binary data to /uploads and generates a unique filename. IssueService builds the Issue entity, and IssueRepository persists it to MySQL.
- **Observer Trigger:** NotificationService generates an alert: "New issue created by Reporter X", targeting System Admins.
- **Response:** HTTP 201 Created with the new Issue ID.

B. Fetching Issues (Role-Aware Data Filtering)

When a user navigates to the Dashboard, React fires GET /api/issues. IssueController checks the JWT to identify the requester and applies role-aware filtering:

- Admin → SELECT * FROM issues
- Developer → SELECT * FROM issues WHERE assignee_id = ?
- Reporter → SELECT * FROM issues WHERE reporter_id = ?

A JSON array of IssueResponse DTOs is returned for React to render.

C. Assigning a Developer

- **Frontend Call:** An Admin selects a Developer from a dropdown and fires PUT /api/issues/{id}/assign with { "developerId": 5 }.

- **Backend Resolution:** @PreAuthorize("hasRole('ADMIN')") ensures only Admins can hit this endpoint. IssueService fetches the issue, updates the assignee foreign key, and saves it.
- **Observer Trigger:** Generates a notification for the assigned developer: "You have been assigned to Issue #12".

D. Resolving a Bug

- **Frontend Call:** The Developer updates the status dropdown to RESOLVED and fires PUT /api/issues/{id}/status with { "status": "RESOLVED" }.
- **Backend Resolution:** The backend verifies the user is either an Admin or the Developer assigned to that task, then updates the status in the database.
- **Observer Trigger:** Generates a notification for the Reporter: "Your bug report #12 has been resolved".

4.3 Collaboration Flow (Comments)

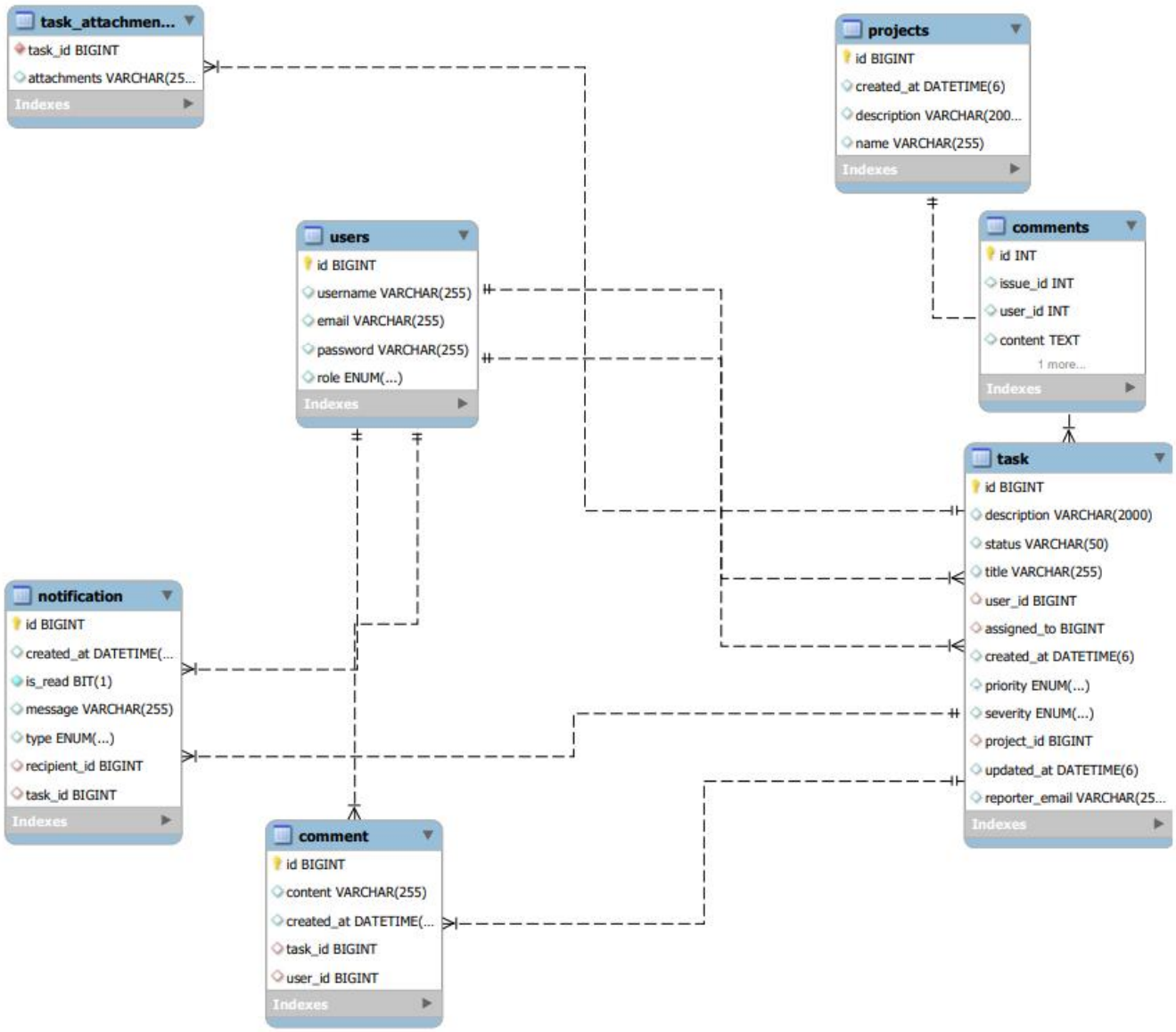
- **Frontend Call:** A user fires POST /api/issues/{id}/comments with { "text": "I found the root cause!" }.
- **Backend Resolution:** The user ID is extracted directly from the JWT, preventing comment spoofing. The Issue ID is verified, and the Comment entity is saved to MySQL with linked User ID and Issue ID foreign keys.
- **Observer Trigger:** Developer comments notify the Reporter; Reporter comments notify the Developer.
- **Response:** Returns the saved comment DTO (with timestamp) so React appends it to the chat UI immediately without a page refresh.

4.4 Admin Analytics Flow

- **Frontend Call:** When an Admin opens the dashboard, React fires GET /api/dashboard/stats.
- **Backend Resolution:** DashboardService runs optimized SQL aggregate functions (e.g., SELECT COUNT(*) FROM issues WHERE status = 'OPEN') to compile metrics including total bugs, resolution percentage, and recent activity logs.
- **Response:** Returns a single JSON object with all analytical metrics. React populates the color-coded statistic cards and resolution progress bar.

5. Database Design & Schema Architecture

TrackIT uses a relational database architecture implemented with MySQL Workbench and MySQL. The database was designed to support the complete workflow of issue tracking, including user management, task handling, collaboration, notifications, and file attachments. The schema follows a normalized relational design to ensure data consistency, reduced redundancy, efficient querying, and maintainable relationships between entities.



5.1 Entity Relationship Overview

The ERD illustrates the relationships between the main entities of the TrackIT system. The task table acts as the central entity, connecting users, projects, comments, notifications, and attachments through foreign key relationships.

5.2 Main Database Entities

Table	Purpose
users	Stores user accounts and roles
projects	Stores project information
task	Stores bug reports and issue tickets
comment	Stores comments on tasks
notification	Stores system-generated alerts
task_attachments	Stores uploaded file references

5.3 Users Table

Stores authentication and authorization information for all registered users. Passwords are stored using BCrypt hashing for security. The role attribute supports the RBAC system implemented in the backend.

Attribute	Description
id	Unique user identifier (Primary Key)
username	User display name
email	Login email address
password	BCrypt-encrypted password
role	User role: ADMIN | DEVELOPER | REPORTER

5.4 Projects Table

Stores information about software projects managed within the system. One project can contain multiple tasks.

Attribute	Description
id	Unique project identifier (Primary Key)

Attribute	Description
name	Project name
description	Project description
created_at	Timestamp of project creation

5.5 Task Table

The core entity of TrackIT. Manages the complete lifecycle of software issues from creation to resolution, and supports task assignments, progress tracking, and dashboard analytics.

Attribute	Description
id	Unique task identifier (Primary Key)
title	Issue title
description	Detailed issue description
status	OPEN | IN_PROGRESS | RESOLVED | CLOSED
priority	LOW | MEDIUM | HIGH
severity	MINOR | MAJOR | CRITICAL
user_id	FK → users (Reporter)
assigned_to	FK → users (Developer)
project_id	FK → projects
created_at / updated_at	Lifecycle timestamps

5.6 Comment Table

Enables collaboration between Reporters, Developers, and Admins by maintaining discussion threads for each task.

Attribute	Description
id	Unique comment identifier
content	Comment text
created_at	Timestamp of comment
task_id	FK → task
user_id	FK → users (Author)

5.7 Notification Table

Stores alerts generated by system events to improve inter-user communication.

Attribute	Description
id	Unique notification identifier
message	Notification message text
type	TASK_ASSIGNED | TASK_UPDATED | TASK_COMPLETED | COMMENT_ADDED | SYSTEM_ALERT
recipient_id	FK → users (Recipient)
task_id	FK → task
is_read	Read/unread flag
created_at	Notification timestamp

5.8 Task Attachments Table

Instead of storing files directly in the database, the system stores files on the server and keeps only file paths in the database. This approach improves storage efficiency and performance.

5.9 Entity Relationships

Relationship	Description
User → Task	One user can create multiple tasks
Developer → Task	One developer can be assigned multiple tasks
Project → Task	One project can contain multiple tasks
Task → Comment	One task can have multiple comments
User → Comment	One user can post multiple comments
Task → Notification	One task can generate multiple notifications
User → Notification	One user can receive multiple notifications
Task → Attachments	One task can have multiple file attachments

All relationships are enforced through foreign-key constraints to maintain referential integrity across the schema.

5.10 Database Design Benefits

- Organized and modular structure with clear separation of concerns
- Efficient task management through normalized relational design
- Strong relational integrity enforced via foreign-key constraints
- Improved collaboration support through linked comment and notification tables
- Secure authentication handling with BCrypt-hashed passwords
- Scalable architecture ready for future feature enhancements

6. Architectural Reasoning & Design Choices

Every major technology decision in TrackIT was made deliberately. This section explains the rationale behind three key architectural choices.

6.1 Monolithic Architecture vs. Microservices

TrackIT was built as a Spring Boot Monolith rather than a Microservices architecture. A bug-tracking system relies heavily on highly coupled, relational data — Users, Issues, and Comments are deeply interdependent. Microservices would introduce network latency between services and require complex distributed transaction management (e.g., Saga patterns) to maintain data integrity. A monolith provides simplicity, easier debugging, and optimal performance by allowing strict ACID compliance within a single, unified database transaction.

Factor	Monolith (Chosen)	Microservices
Complexity	Low — single deployable unit	High — service coordination required
Data integrity	Native ACID transactions	Distributed transactions (Saga/2PC)
Debugging	Straightforward local debugging	Requires distributed tracing
Scalability	Vertical / whole-unit horizontal	Granular per-service scaling

6.2 JWT vs. Session-Based Authentication

Stateless JSON Web Tokens (JWT) were selected over traditional stateful server sessions. With session authentication, the backend must store and look up a session ID on every request — creating a performance bottleneck and memory overhead. With JWTs, the cryptographic signature carries all necessary authorization data (user ID and role), enabling the server to validate requests mathematically without any database lookup.

Factor	JWT (Chosen)	Session-Based
State	Stateless — self-contained token	Stateful — server stores sessions
Scalability	Horizontal scaling friendly	Requires sticky sessions / shared cache
Performance	No DB lookup per request	DB/cache lookup on every request
Revocation	Harder — token valid until expiry	Immediate — delete session record

6.3 React Context API vs. Redux

The React Context API was used for global state management (user authentication and notifications) instead of Redux. Since TrackIT's global state is relatively simple and does not require complex time-travel debugging or extensive middleware chains, Context API provides a cleaner, lighter, and more maintainable solution without Redux's boilerplate overhead.

7. Trade-offs & System Limitations

Every architectural decision involves trade-offs. To maintain scope while delivering high performance, the following limitations were consciously accepted.

7.1 Local File Storage vs. Cloud Storage

Aspect	Detail
Approach	File attachments are stored on the local server filesystem (/uploads)
Pros	Zero financial cost, fast local read/write speeds, simple implementation
Cons	Files are lost if server hardware fails; horizontal scaling is prevented as additional servers would not share the same filesystem
Future Mitigation	Migration to a cloud storage provider such as AWS S3 or Azure Blob Storage

7.2 Monolithic Scalability Limits

Aspect	Detail
Approach	A single Spring Boot application handles all routing, logic, and notifications
Pros	Extremely easy to deploy and test locally
Cons	Individual components (e.g., the Notification Engine) cannot be scaled independently; the entire monolith must be scaled, which is resource-inefficient under targeted load spikes
Future Mitigation	Selective extraction of high-load components into separate services

7.3 JWT Security Risks (localStorage Exposure)

Aspect	Detail
Approach	JWTs are stored in the browser's localStorage for stateless session management
Pros	Eliminates database lookups for authentication, making the API highly performant
Cons	A JWT cannot be easily revoked before expiry; if stolen via a Cross-Site Scripting (XSS) attack, the attacker retains full access until token expiration
Mitigation Applied	Strict input sanitization across all React forms to prevent XSS vulnerabilities
Future Mitigation	Use HttpOnly cookies to store tokens, preventing JavaScript access

— End of Report —